

세션 3 — 설계 & 환경 구축

이번 세션 종료 시 본인 리포지토리는

`git clone` → 한 줄 명령으로 실행 가능해야 합니다.

오늘의 목표

1. 강의 슬라이드를 페이지씩 보며 **큰 그림 이해**
2. 모바일 앱 플랫폼 선택과 그 이유를 **ADR로 말하고 기록**
3. 아키텍처 다이어그램과 디렉토리 구조 확정
4. **빌드되는 "Hello World" 상태까지 만들기**
5. `docs/setup.md`, `docs/architecture.md` 작성
6. 실습 중 3회 점검으로 진행 상태 확인

오늘의 진행 방식

```
<svg viewBox="0 0 980 270" width="100%" role="img" aria-label="강의 진행 방식">
<defs> <linearGradient id="g1" x1="0" x2="1"> <stop offset="0" stop-
color="#2563eb"/> <stop offset="1" stop-color="#7c3aed"/> </linearGradient> <filter
id="shadow" x="-20%" y="-20%" width="140%" height="140%"> <feDropShadow
dx="0" dy="8" stdDeviation="8" flood-color="#0f172a" flood-opacity="0.18"/>
</filter> </defs> <rect x="30" y="40" width="270" height="170" rx="24" fill="#eff6ff"
stroke="#2563eb" stroke-width="3" filter="url(#shadow)"/> <rect x="355" y="40"
width="270" height="170" rx="24" fill="#f5f3ff" stroke="#7c3aed" stroke-width="3"
filter="url(#shadow)"/> <rect x="680" y="40" width="270" height="170" rx="24"
fill="#ecfdf5" stroke="#059669" stroke-width="3" filter="url(#shadow)"/> <path
d="M310 125 H345" stroke="url(#g1)" stroke-width="8" stroke-linecap="round"/>
<path d="M635 125 H670" stroke="#059669" stroke-width="8" stroke-
linecap="round"/> <text x="165" y="92" text-anchor="middle" font-size="30" font-
```

페이지별로 볼 때의 질문 3개

각 슬라이드는 아래 3문장으로 정리합니다.

```
flowchart LR
    A[이 슬라이드의 핵심 개념은?] --> B[우리 프로젝트에 왜 필요한가?]
    B --> C[오늘 어떤 산출물로 남길 것인가?]
    C --> D[README / docs / ADR / 코드]
```

관점	학생이 답해야 할 말
개념	"이건 무엇인가?"
이유	"우리 앱에서 왜 필요한가?"
산출물	"어느 파일/문서에 남길 것인가?"

1. 플랫폼 선택지

플랫폼	장점	단점	추천 대상
Flutter	한 코드로 양 OS, UI 풍부	Dart 학습	빠른 프로토타입
React Native	JS/TS 생태계, 익숙함	네이티브 모듈 시 복잡	웹 경험자
Android(Kotlin)	풀 네이티브, 도구 성숙	iOS 별도	Android만 타겟
iOS(Swift)	Apple 생태계 깊이	macOS 필수	iOS만 타겟
Kotlin Multiplatform	코어 공유, 네이티브 UI	복잡도 높음	중급 이상

→ 선택 후 ADR-0001: 모바일 플랫폼 선택으로 기록

ADR로 말하기 — 이번 주 내용을 꺼내기

첫 번째 실습 점검은 **이번 주 ADR을 이용한 말하기 훈련**입니다.

```
<svg viewBox="0 0 980 300" width="100%" role="img" aria-label="ADR 말하기 카드">
<defs> <linearGradient id="adr" x1="0" x2="1" y1="0" y2="1"> <stop offset="0" stop-
color="#fef3c7"/> <stop offset="1" stop-color="#fde68a"/> </linearGradient>
</defs> <rect x="40" y="35" width="900" height="225" rx="26" fill="url(#adr)"
stroke="#f59e0b" stroke-width="4"/> <text x="90" y="92" font-size="32" font-
weight="800" fill="#78350f">ADR 60초 말하기 카드</text> <text x="90" y="145" font-
size="24" fill="#0f172a">1. 우리는 무엇을 선택했나?</text> <text x="90" y="185" font-
size="24" fill="#0f172a">2. 다른 선택지는 무엇이었나?</text> <text x="90" y="225"
font-size="24" fill="#0f172a">3. 우리 팀의 기준에서 왜 이 선택이 더 낫나?</text> <circle
cx="820" cy="145" r="58" fill="#fff7ed" stroke="#ea580c" stroke-width="4"/> <text
x="820" y="136" text-anchor="middle" font-size="30" font-weight="800"
fill="#9a3412">60초</text> <text x="820" y="171" text-anchor="middle" font-
```

2. 아키텍처 — 가장 흔한 모바일 앱 레이어

flowchart TB

```
P[Presentation<br/>UI · Screen · View] --> A[Application<br/>ViewModel · UseCase]
A --> D[Domain<br/>Entity · Service · Rule]
D --> R[Data<br/>Repository · API · DB]
```

```
classDef ui fill:#dbeafe,stroke:#2563eb,color:#0f172a,stroke-width:2px;
classDef app fill:#ede9fe,stroke:#7c3aed,color:#0f172a,stroke-width:2px;
classDef domain fill:#dcfce7,stroke:#16a34a,color:#0f172a,stroke-width:2px;
classDef data fill:#fee2e2,stroke:#dc2626,color:#0f172a,stroke-width:2px;
class P ui;
class A app;
class D domain;
class R data;
```

AI Agent에게 "이 4개 레이어로 우리 프로젝트 구조를 잡아줘"
라고만 시키면 95%는 정확하게 만듭니다.

3. 디렉토리 구조 (Flutter 예시)

```
lib/  
├── main.dart  
├── app.dart  
├── presentation/  
│   ├── screens/  
│   ├── widgets/  
│   └── theme/  
├── application/  
│   └── view_models/  
├── domain/  
│   ├── entities/  
│   └── services/  
└── data/  
    ├── repositories/  
    ├── api/  
    └── local/
```


아키텍처를 파일 위치로 바꾸기

```
<svg viewBox="0 0 980 330" width="100%" role="img" aria-label="레이어와 파일 위치
매핑"> <style> .title { font: 700 24px sans-serif; fill: #0f172a; } .body { font: 18px sans-
serif; fill: #334155; } .small { font: 16px sans-serif; fill: #475569; } </style> <rect x="35"
y="35" width="205" height="230" rx="20" fill="#dbeafe" stroke="#2563eb" stroke-
width="3"/> <rect x="275" y="35" width="205" height="230" rx="20" fill="#ede9fe"
stroke="#7c3aed" stroke-width="3"/> <rect x="515" y="35" width="205" height="230"
rx="20" fill="#dcfce7" stroke="#16a34a" stroke-width="3"/> <rect x="755" y="35"
width="190" height="230" rx="20" fill="#fee2e2" stroke="#dc2626" stroke-
width="3"/> <text x="138" y="80" text-anchor="middle"
class="title">Presentation</text> <text x="138" y="125" text-anchor="middle"
class="body">screens/</text> <text x="138" y="158" text-anchor="middle"
class="body">widgets/</text> <text x="138" y="191" text-anchor="middle"
class="body">theme/</text> <text x="378" y="80" text-anchor="middle"
class="body">components/</text> <text x="578" y="80" text-anchor="middle"
class="body">animations/</text> <text x="778" y="80" text-anchor="middle"
class="body">utils/</text> <text x="978" y="80" text-anchor="middle"
class="body">hooks/</text> </svg>
```

4. 환경 구축 체크리스트

항목	확인
Git 저장소 생성	✓
<code>.gitignore</code> (플랫폼별)	✓
README — 프로젝트 한 줄 설명	✓
의존성 명세 (<code>pubspec.yaml</code> 등)	✓
첫 빌드 성공	✓
<code>docs/setup.md</code> 작성	✓
에디터 권장 확장 (<code>.vscode/extensions.json</code>)	✓

5. docs/setup.md 의 절대 조건

새 사람이 이 문서만 보고 5분 안에 실행할 수 있어야 한다.

포함되어야 할 내용

1. 필요한 도구 버전 (Node 20.x, Flutter 3.x, JDK 17 등)
2. 클론 명령
3. 의존성 설치 명령
4. 환경 변수 설정 방법 (`.env.example` 포함)
5. 첫 실행 명령
6. 문제 해결 (FAQ 5개)

AI Agent에게 "윈도우/맥/리눅스 모두 따라할 수 있게" 명시 요청

실습 운영 타임라인 — 3회 확인

```
gantt
  title Week 11 실습 진행 리듬
  dateFormat HH:mm
  axisFormat %H:%M
  section 강의
  슬라이드 페이지별 이해           :a1, 00:00, 00:25
  section 실습
  점검 1 ADR 말하기                 :milestone, m1, 00:35, 0m
  구조/다이어그램/폴더 작성         :a2, 00:25, 00:45
  점검 2 구조 확인                  :milestone, m2, 01:10, 0m
  Hello World 빌드                  :a3, 01:10, 00:35
  점검 3 실행·문서·Push 확인        :milestone, m3, 01:45, 0m
```

시간은 예시입니다. 실제 수업에서는 학생 진행 속도에 맞춰 3번만 확실히 멈추기.

점검 1 — ADR로 말할 수 있는가?

확인 시점: 실습 초반, 플랫폼/상태관리/백엔드 선택을 시작하기 전

확인 질문	좋은 답변의 조건
지난주 ADR의 결정은 무엇인가?	선택 항목을 한 문장으로 말함
대안은 무엇이었나?	최소 1개 이상 비교함
왜 그 결정을 유지/변경하나?	팀 프로젝트 기준과 연결함

학생 말하기 템플릿

"우리 팀은 [선택] 을 선택했습니다.
대안으로는 [대안] 이 있었지만, [우리 기준] 때문에 현재 선택이 더 적합합니다.
그래서 오늘은 이 결정을 ADR-0001 에 반영하고 구조를 만들겠습니다."

점검 2 — 구조를 설명할 수 있는가?

확인 시점: 디렉토리 구조와 `docs/architecture.md` 초안 작성 후

```
flowchart LR
    Q1{새 화면은?} --> P[presentation/screens]
    Q2{상태/흐름은?} --> A[application/view_models]
    Q3{핵심 규칙은?} --> D[domain]
    Q4{API/DB는?} --> R[data]
```

- 파일이 "대충 생겼다"가 아니라 **위치 이유**를 설명해야 합니다.
- `docs/architecture.md` 에 Mermaid 다이어그램이 있어야 합니다.
- AI Agent가 만든 구조라도 학생이 읽고 수정할 수 있어야 합니다.

점검 3 — 실행과 인수인계가 되는가?

확인 시점: 수업 후반, 빌드/실행/문서/Push 직전

```
<svg viewBox="0 0 980 260" width="100%" role="img" aria-label="clone to run
pipeline"> <defs> <linearGradient id="pipe" x1="0" x2="1"> <stop offset="0" stop-
color="#06b6d4"/> <stop offset="1" stop-color="#22c55e"/> </linearGradient>
</defs> <path d="M110 130 H870" stroke="url(#pipe)" stroke-width="18" stroke-
linecap="round"/> <g fill="#ffffff" stroke="#0f172a" stroke-width="3"> <circle
cx="145" cy="130" r="58"/> <circle cx="330" cy="130" r="58"/> <circle cx="515"
cy="130" r="58"/> <circle cx="700" cy="130" r="58"/> <circle cx="855" cy="130"
r="58"/> </g> <g font-family="sans-serif" text-anchor="middle" fill="#0f172a"> <text
x="145" y="122" font-size="20" font-weight="700">clone</text> <text x="145"
y="150" font-size="15">받기</text> <text x="330" y="122" font-size="20" font-
weight="700">install</text> <text x="330" y="150" font-size="15">설치</text> <text
x="515" y="122" font-size="20" font-weight="700">env</text> <text x="515" y="150"
```

9. AI Agent 활용 — 이번 주 명령

- 1) 우리 프로젝트의 디렉토리 구조를 레이어드 아키텍처로 만들어줘
- 2) docs/architecture.md 에 시스템 다이어그램(Mermaid) 포함
- 3) docs/setup.md 에 zero → run 절차 작성
(윈도우/맥/리눅스 모두)
- 4) 아무 화면이나 1개를 만들고 빌드 성공 확인
- 5) ADR-0001 ~ 0003 생성
(프레임워크, 상태관리, 백엔드 선택 이유)

AI Agent에게 맡기되, 학생이 검수하기

```
sequenceDiagram
    participant S as Student
    participant A as AI Agent
    participant R as Repository
    participant D as Docs/ADR

    S->>A: 구조/문서/초기 화면 생성 요청
    A->>R: 파일 생성 및 수정
    A->>D: Mermaid 다이어그램·setup 작성
    S->>R: 빌드/실행 확인
    S->>D: 결정 이유와 실행 절차 검수
    S-->>A: 틀린 부분 수정 요청
```

핵심: AI가 만들고, 학생이 설명하고 검수합니다.

10. "이해했다"의 정의 — 이번 주

본인이 다음에 답할 수 있어야 합니다.

- [] 지난주 ADR의 결정을 60초 안에 말할 수 있는가?
- [] 우리 프로젝트의 디렉토리 구조를 머리로 그릴 수 있는가?
- [] 새 화면을 추가하려면 어디에 파일을 만들어야 하나?
- [] API 호출은 어느 레이어에서 일어나는가?
- [] 빌드가 실패하면 어디부터 봐야 하나?
- [] 왜 이 프레임워크를 선택했나? (대안 vs 이유)
- [] `git clone` 후 한 줄 명령으로 실행되는가?

11. 다음 세션까지

1. `docs/architecture.md`, `docs/setup.md` 작성 완료
2. ADR 최소 3개 작성
3. "Hello World" 빌드 성공 push
4. (2인 팀) 두 사람 모두 로컬에서 빌드 성공 인증
5. 다음 주 **중간 발표** 준비 시작 — 슬라이드 초안

함께 보는 자료

- `resources/01-platform-comparison.md` — 플랫폼 상세 비교
- `resources/02-architecture-patterns.md` — 모바일 앱 아키텍처 패턴
- `resources/03-setup-template.md` — `docs/setup.md` 템플릿
- `적용지침.md` — 12/13/14주차에도 반복 적용하는 운영 지침